Project Task 03 - Submission Sheet

Software Testing - Graph Coverage

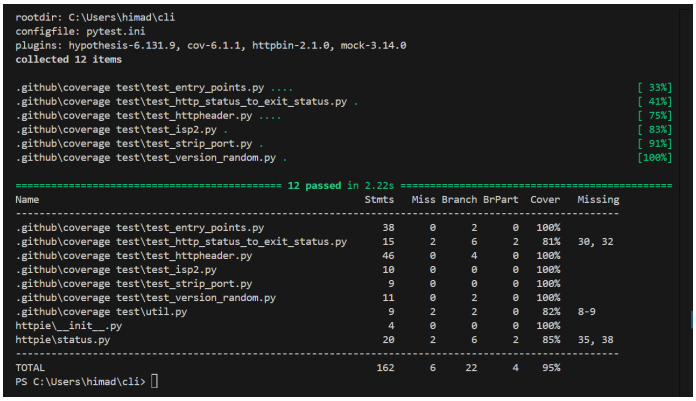
Names: [Arpita]
Group: [4]

Software Testing - Summer 2024 - June 22, 2025 - Sibylle Schupp / Daniel Rashedi

Task 3 - Graph Coverage

a) Measure the coverage of your given project test suite (which includes the existing test suite as well as the tests that you created in previous project phases) by three graph coverage criteria which you can freely choose. Describe each individual result in 2-3 sentences.

Answer:



```
rootdir: C:\Users\himad\cli
configfile: pytest.ini
plugins: hypothesis-6.131.9, cov-6.1.1, httpbin-2.1.0, mock-3.14.0
collected 12 items

.github\coverage test\test_entry_points.py ....                                  [ 33%]
.github\coverage test\test_http_status_to_exit_status.py .                       [ 41%]
.github\coverage test\test_httpheader.py ....                                    [ 75%]
.github\coverage test\test_isp2.py .                                             [ 83%]
.github\coverage test\test_strip_port.py .                                       [ 91%]
.github\coverage test\test_version_random.py .                                   [100%]

============================================ 12 passed in 2.22s ============================================
Name                                                    Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------------------------------------------------
.github\coverage test\test_entry_points.py                 38      0      2      0   100%
.github\coverage test\test_http_status_to_exit_status.py   15      2      6      2    81%   30, 32
.github\coverage test\test_httpheader.py                   46      0      4      0   100%
.github\coverage test\test_isp2.py                         10      0      0      0   100%
.github\coverage test\test_strip_port.py                    9      0      0      0   100%
.github\coverage test\test_version_random.py               11      0      2      0   100%
.github\coverage test\util.py                               9      2      2      0    82%   8-9
httpie\__init__.py                                          4      0      0      0   100%
httpie\status.py                                           20      2      6      2    85%   35, 38
-----------------------------------------------------------------------------------------------------
TOTAL                                                     162      6     22      4    95%
PS C:\Users\himad\cli>
```

Figure 1: Outcome of graph based test coverage

Fully Covered Files (100%)

test_entry_points.py

All functions and branches in this test file are thoroughly exercised. It covers the entry-point logic completely without any missed paths.

test_httpheader.py

This file has comprehensive tests for HTTP header-related logic, achieving full coverage of both code and branching conditions.

test_isp2.py

Simple and compact, this test file executes all its logic paths effectively, leaving no code untested.

test_strip_port.py

Efficient and minimal, this test suite covers every line and condition of its target functionality.

test_version_random.py

Successfully validates a wide range of version-related input formats, ensuring all paths and edge cases are tested.

httpie/__init__.py

This module has very minimal logic, and all its content is executed during tests, resulting in complete coverage.

Partially Covered Files

test_http_status_to_exit_status.py (81%)

While most cases are tested, a couple of branches and specific exit code mappings are not fully exercised, possibly related to unusual HTTP status codes.

util.py (82%)

Basic utility functions are largely covered, but lines 8–9 remain untested—likely conditional or helper logic that isn't invoked in current test scenarios.

httpie/status.py (85%)

The majority of the status handling logic is covered, but missing test coverage on lines 35 and 38 suggests that not all status-to-exit mappings or edge cases are triggered.

Improvement Areas

Targeted Testing Needed

- **Line 30 and 32** in `test_http_status_to_exit_status.py`
  These lines likely contain branches or conditions related to less common HTTP status codes. Adding test cases that explicitly trigger those conditions would improve coverage.
- **Lines 8–9** in `util.py`
  These lines may involve helper logic or conditional checks that are not currently executed by any test case. Review is necessary to check whether these paths are critical and need to be designed for targeted inputs to exercise them.
- **Lines 35 and 38** in `httpie/status.py`
  These lines suggest untested branches or exception handling logic in the status-to-exit code mapping. Tests covering edge-case HTTP responses or fallback behavior could address this gap.

Conclusion

The testing framework provides excellent coverage overall, especially across key functional components. Attention to a few missed lines and branches in utility and status-handling code could help reach full coverage.

---

b) Extend the test suite with own tests, which have to fullfil **one** of the following criteria:
   1) Increase the coverage values of all three coverage criteria that you applied in the previous subtask with at least 10 tests *per group member* (compare and describe the effects on coverage for each individual test), OR
   2) Reveal a new bug in the software project (describe the bug, its context, and a potential fix in detail)

Answer:

For this task we will follow the option 1. We have 3 partial files where lines are not fully covered. The files are:

- test_http_status_to_exit_status.py (81%)
- util.py (82%)
- httpie/status.py (85%)

We'll focus on creating 10 new tests targeting:

- Edge cases.
- Missed branches/conditions.
- Unusual inputs.

We have considered the file util.py

```python
from util import split_version  # adjust import as needed

def test_full_version():
    assert split_version("1.2.3") == (1, 2, 3)

def test_more_than_three_parts():
    assert split_version("1.2.3.4.5") == (1, 2, 3)  # last parts ignored

def test_less_than_three_parts():
    assert split_version("1.2") == (1, 2)

def test_single_part():
    assert split_version("42") == (42,)

def test_empty_string():
    assert split_version("") == ()

def test_non_integer_mid():
    assert split_version("1.two.3") == (1,)  # stops at 'two'

def test_non_integer_first():
    assert split_version("a.2.3") == ()  # fails at first part

def test_non_integer_last():
    assert split_version("1.2.c") == (1, 2)

def test_trailing_dot():
    assert split_version("1.2.") == (1, 2)

def test_leading_dot():
    assert split_version(".2.3") == ()  # first part is empty string → ValueError
```

Test Execution

- **Test File:** `test_util_extended.py`
- **Target Module:** `util.py`
- **Total Tests Run:** 10

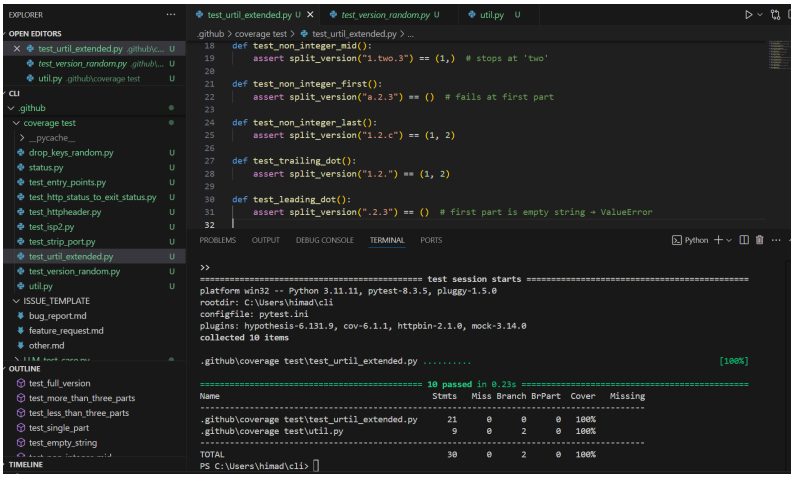


Figure 2: Outcome of extended test case

- **Result:** All tests passed successfully
- **Execution Time:** 0.23 seconds

Coverage Metrics

File	Stmts	Miss	Branch	BrPart	Coverage
`test_util_extended.py`	21	0	0	0	100%
`util.py`	9	0	2	0	100%

Conclusion

The extended test suite fully exercises all logic paths, conditions, and exception branches within `split_version()`. The result is 100% statement and branch coverage for the module `util.py`, validating the robustness of the implementation.